EXECUTIVE ENGINEERING REPORT & PRESENTATION

Global Multi-Language Business Card Maker — Web, iOS & Android

How I Built a 300 DPI Print Studio, Unified 11 Global Languages with IP Auto-Switching, and Compiled Native iOS (.ipa) & Android (.apk) Apps from a Windows PC

Author: Iqbal Mahmud (AiInfos) **App ID:** com.aiinfos.businesscardmaker **Verification:** 138 / 138 E2E Tests Passing

148 Designs
8 Classic + 140 Procedural Vector Layouts (35 Styles)

11 Languages
Unified Engine with Bangla Typography & Arabic RTL

300 DPI PDF
2-Page Print Shop Export with 3 mm Bleed & vCard QR

iOS + Android
100% Offline Mobile Apps Built via GitHub Actions

01 Executive Overview & Business Impact

I designed and engineered a print-shop-grade **Business Card Maker** that runs seamlessly both in web browsers and as a **100% offline native iOS (`.ipa`) and Android (`.apk` / `.aab`) mobile app**. Because all vector rendering and binary PDF generation happen directly on the user's device, the platform achieves commercial SaaS quality with **$0 recurring server or database cost** and complete user privacy.

Capability	Technical Implementation	Business Value
Print-Ready Output	Custom HTML5 Canvas 2D vector renderer + custom binary PDF 1.4 writer at 300 DPI	Ready for professional offset & digital print shops (3 mm bleed + trim/safe-zone guides)
Global Localization	11 languages unified in one codebase + non-blocking Geo-IP auto-detection	Immediate international adoption across Asia, Europe, Americas, and MENA
Cross-Platform Mobile	Capacitor 6 + 146 bundled offline `.woff2` fonts + GitHub Actions CI/CD	Single codebase ships to Web, iPhone/iPad (`.ipa`), and Android (`.apk`) from Windows

02 Core Studio Architecture (300 DPI PDF, vCard QR & Digital Wallets)

- **148 Vector Card Designs & 20 Color Palettes:** Engineered 8 classic layouts plus 140 procedural designs across 35 visual families (waves, gold frames, botanical, geometric, circuit, etc.), supporting US (`3.5 × 2 in`), European (`85 × 55 mm`), and Australian/Indian (`90 × 55 mm`) card sizes.
- **Zero-Dependency Binary PDF 1.4 Generator (`src/pdf-writer.js`):** Exports a 2-page (Front & Back) 300 DPI PDF with embedded `/TrimBox` and `/BleedBox` metadata directly in memory without external libraries.
- **Smart Contact QR & Social Links:** Encodes UTF-8 `vCard 3.0` contact cards with 7 social platforms (LinkedIn, X, Facebook, Instagram, YouTube, TikTok, GitHub) into the QR code, plus **Apple Wallet**, **Google Wallet**, and `.vcf` contact export.
- **Cryptographic Content-Security-Policy (CSP):** Computes SHA-256 hashes of inline scripts at build time so no unauthorized script can execute.

03 Unifying Bangla + 11 Global Languages in a Single Codebase

Instead of maintaining separate repositories for different languages, I consolidated the English and Bangla editions into one unified multi-language engine (`src/i18n/locales.js`) and expanded support to **11 languages**:

> **Supported Languages (11):** English (en) · বাংলা (bn) · Français (fr) · Deutsch (de) · Español (es) · हिन्दी (hi) · 日本語 (ja) · 简体中文 (zh) · 繁體中文 (zh-Hant) · العربية (ar — RTL) · Русский (ru)

- **Full Bangla Typography & Numeral Parity:** Preserved all 6 curated Bengali font pairings (*Noto Sans Bengali*, *Noto Serif Bengali*, *Hind Siliguri*, *Anek Bangla*, *Baloo Da 2*, *Tiro Bangla*, *Galada*), Bengali numeral formatting (০–৯), and full Bangla localization of all 148 design names, 20 color palettes, UI controls, and the About & FAQ guide.
- **Dynamic RTL/LTR Layout Switching:** Selecting Arabic (`ar`) automatically flips the document layout to Right-to-Left (`dir="rtl"`) while keeping card canvas coordinates accurate.

04 Smart IP-Based Automatic Language Switching

To greet every visitor in their local language without slowing down page load, I implemented a **3-tier non-blocking language detection pipeline**:

1. User Priority
Checks `?lang=` URL parameter or saved manual choice in `localStorage`.

2. Geo-IP Lookup
Queries `api.country.is` (with `get.geojs.io` failover, 1.5s timeout & 6h cache).

3. Country Mapping
Maps 45+ country codes (e.g., `BD → bn`, `FR → fr`, `DE → de`, `SA → ar`, `JP → ja`).

4. Polite Toast
Shows a dismissible localized notice when auto-switching; manual choice always wins.

05 Upgrading to a True Native Mobile App Experience

- **Full-Screen "Present Card / QR" Mode (`#presentModal`):** One tap on *Present* opens a full-screen card & scannable vCard QR view for in-person meetings, with quick *Share PDF* and *Save contact (.vcf)* actions.
- **"My Cards" Multi-Card Deck (`#deckWrap`):** Users can save, update, switch between, and delete up to **25 different business cards** on their device (e.g., Personal, Work, Bangla version, English version).
- **100% Offline Font Bundling (`build/build_mobile.py`):** Bundles all **146 Latin and Bengali `.woff2` font files** (27 font families) directly inside `dist/mobile/fonts/` so the app works completely offline (even in Airplane Mode).
- **Native Share Sheet & Haptics:** Routes PDF, PNG, and `.vcf` exports through `@capacitor/filesystem` + `@capacitor/share` (AirDrop, Save to Files, WhatsApp, Print) and triggers native tactile haptics via `@capacitor/haptics`.

06 How I Built the iOS (.ipa) & Android (.apk) Apps from a Windows PC

The Challenge: Apple requires macOS and Xcode to compile iOS apps, which normally requires purchasing Mac hardware.

The Solution: I engineered an automated **Capacitor 6 + GitHub Actions Cloud CI/CD Pipeline** (`.github/workflows/mobile-apps.yml`) driven directly from my Windows PC:

1. Windows PC
Author code, run `build_mobile.py`, and verify 138 Playwright E2E checks locally.

2. GitHub Push
Push to GitHub and trigger the parallel *Build iOS & Android Mobile Apps* workflow.

3. Cloud Builders
`ubuntu-latest` (Java 21/Gradle) builds Android; `macos-latest` (Xcode) builds iOS.

4. Ready Packages
Outputs `BusinessCardMaker-debug.apk` & `BusinessCardMaker-unsigned.ipa` (+ optional signed builds).